

ANÁLISIS DE OBSERVABILIDAD · CONFIDENCIAL

Evaluación de Logging y Observabilidad de Aplicaciones

Clúster **SAT** — comenzando por **dev**, con clúster de referencia **GLX UAT**

Proyecto	Tigo Panamá — Zeus
Entorno	SAT dev (referencia: GLX UAT)
Fecha del análisis	23 de junio de 2026
Estado	Evaluación / propuesta — nada aplicado en ningún clúster
Identificación	Ing = Longinus · pa = Panamá

Logs de aplicación primero · Service mesh y Kibana en fases posteriores

1. Resumen ejecutivo

El logging de SAT se recolecta correctamente, pero no es de calidad para soporte: texto plano, nivel DEBUG efectivo y dominado por ruido de sondas. La buena noticia es que el comportamiento de logging está centralizado en las imágenes base compartidas, por lo que puede estandarizarse en una sola capa, sin tocar el código de ninguna aplicación.

Lo que más importa	Conclusión
Service mesh	Ni SAT ni GLX tienen Istio ni Kiali. Un mesh en SAT es una instalación greenfield , no "copiar GLX".
Aislamiento de clústeres	SAT y GLX están totalmente aislados (sin ruta de red). El ELK externo de GLX no puede reutilizarse: SAT necesita su propio backend de logs.
Formato de logs	Todos los logs son texto plano — cero JSON en ningún lado. La recolección funciona (SAT: Promtail→Loki; GLX: fluent-bit→ELK externo), pero los logs no son de calidad para soporte : DEBUG efectivo, dominados por ruido de sondas, sin estructura consistente.
Hallazgo central	El comportamiento de logging está centralizado en las imágenes base compartidas (<code>tigo-spring-base</code> , <code>tigo-python-base</code> , <code>tigo-php-base</code>). Se puede estandarizar en la capa de imagen base — un solo cambio, heredado por todas las apps al reconstruir — sin editar el código fuente de ninguna aplicación . La base Spring (<code>sit-1.1.0</code>) ya implementa un <code>CorrelationIdFilter</code> (genera/propaga un <code>correlation_id</code>).
Restricción respetada	No se cambia código de aplicación. Dos vías complementarias y libres de código (L1 y L2).
Decisión	Backend de logs de SAT = Elastic + Kibana propios, en el clúster vía el operador ECK . Logs y el plan de Istio/Kiali corren en paralelo , logs primero.

Dos vías complementarias, ambas sin cambios de código

L1 — PIPELINE Sin reconstrucción. Normalizar en captura en **fluent-bit**: parsers por formato, unión de stack traces multilínea, descarte de ruido de sondas, índice por namespace.

L2 — IMAGEN BASE Reconstrucción, sin cambiar código fuente. Actualizar las imágenes `tigo-*-base` para emitir **JSON**, usar **INFO** por defecto, excluir `/health` de los logs y asegurar el filtro de correlación en todas. Las apps lo heredan en su próximo build de CI.

2. Método

Recolección de **solo lectura** de ambos clústeres a través del bastión (kubeconfig de rke2 en `/etc/rancher/rke2/rke2.yaml`). Se capturó: inventario de clúster/namespace/workload, presencia de mesh y CRDs, los stacks de logging, la configuración de fluent-bit de GLX, y un **relevamiento de formato de log por aplicación** en SAT dev (20 apps representativas en Java/Spring, Python, PHP, nginx y consumidores). **No se aplicó ningún cambio.**

3. Estado actual — SAT dev

3.1 Workloads

- ~30+ apps de negocio en el namespace `argocd` (antipatrón: los workloads comparten el namespace de ArgoCD + RabbitMQ — sin aislamiento de radio de impacto).

- Nomenclatura: Deployment `...-v1`, Service `...-v1-services`, contenedor `app-container-...-v1`. **Un contenedor por pod, sin sidecars.**

3.2 Service mesh

Ninguno. Sin `istio-system`, sin CRDs `istio.io`, sin namespaces con `istio-injection`, sin Kiali, sin sidecars `istio-proxy`.

3.3 Stack de logging

- Namespace `observability`: **Loki + Grafana + Promtail** (DaemonSet, los 13 nodos). Promtail ya recolecta **todo el stdout de los pods** → **Loki**. La recolección no es el problema; la calidad sí.
- Sin Elasticsearch / Kibana en ningún lugar de SAT.

4. Estado actual — GLX UAT (referencia)

- **Service mesh: ninguno** (igual que SAT). Si existe un mesh, sería en GLX **prod** — por confirmar.
- **Logging — el patrón reutilizable:** namespace `logging` con un **DaemonSet de fluent-bit** (13 nodos, imagen `registry-local.tigo.com.pa/fluent-bit:3.1.4`) → **Elasticsearch externo** (secret `elasticsearch-credentials`). Configuración confirmada:
 - **Input:** `tail /var/log/containers/*.log`, parser `cri` (containerd/rke2), DB de offset, `Mem_Buf_Limit 10MB`.
 - **Filter:** `kubernetes` estándar (metadata de pod/namespace) + **allow-list de namespaces en Lua** — solo se envían `cdr-pa`, `glx`, `argocd`, `cattle-system`; el resto se descarta.
 - **Output:** `es` → `elk-mnt.tigo.com.pa:443`, `tls On` / `tls.verify Off`, credenciales por env (`ES_HTTP_USER` / `ES_HTTP_PASSWD`), `Logstash_Format On` con prefijo por namespace (`DateFormat %Y.%m.%d`), `Generate_ID On`, `Replace_Dots On`, `Retry_Limit False`. **Un índice por namespace** `fluent-bit-glx-uat-<ns>-YYYY.MM.DD`.
 - **Parsers definidos:** solo `cri`, `json`, `docker`.

🔑 GLX NO normaliza los logs de aplicación

No hay **parser por aplicación** ni **regla multilínea** — el parser CRI solo separa `time/stream/message`, por lo que en Kibana el `message` es la **línea de texto plano cruda**: sin campos `level` / `logger` / `corr_id`, y los stack traces de Java quedan partidos en muchos documentos. GLX nos da un **esqueleto probado, no una solución de normalización**.

- Namespace `monitoring` = kube-prometheus-stack (métricas).
- **Conclusión:** reutilizar el **esqueleto** de fluent-bit (input CRI, output ES, filtro k8s, allow-list, índice por namespace, la imagen 3.1.4 ya espejada internamente); **cambiar el destino** (SAT → ES propio) y **agregar el parseo/multilínea L1 que GLX no tiene** (§6).

5. Análisis de formato de logs (relevamiento SAT dev)

Cada app emite texto plano a stdout — sin JSON. Se encontraron cinco formatos distintos, todos determinados por el stack / la imagen base:

Formato	Origen y apps
A — Spring "Satélite" (logback custom)	Opcionalmente con campo de correlación en la base más reciente. Apps: <code>cr-*</code> , <code>iuc-*</code> , <code>adminntt-*</code> , <code>billing</code> , <code>tigo-spring-base</code> .
B — Spring Boot consola por defecto	Apps: <code>payments-*</code> (incl. variante WebFlux con <code>[reqId]</code>), <code>emailqueue-consumer</code> , <code>mainconsumer</code> , <code>tytan-tecrep-consumer</code> .
C — Python / FastAPI (uvicorn)	Acceso uvicorn + avisos de arranque multilínea (pydantic). Apps: <code>tigo-python-base</code> , <code>ivrprocessapi</code> .
D — PHP base (custom)	Principalmente pings de <code>/health</code> . Apps: <code>tigo-php-base</code> .
E — nginx access (frontends)	Apps: <code>cr-app-frontend</code> , <code>adminntt-app-frontend</code> .

Subpatrón de logging de negocio (consumidores de notificación): `smsqueue-consumer` y `mainconsumer` usan un logger `LogEntry` con el formato `ApiName | UTI | Timestamp | Comment` — el **UTI** es efectivamente un id de transacción/correlación ya presente en el cuerpo del mensaje.

Matriz por aplicación

App	Imagen / base	Stack	Formato	Niveles	Corr ID	Contenido / ruido dominante
cr-service-getpayments-v1	cash-register	Spring (satélite)	A	DEBUG	no	~100% ruido de sonda /health DEBUG
cr-service-processpayments-v1	cash-register	Spring	A	DEBUG	no	~100% ruido /health DEBUG
iuc-service-processsap-v1	iuc	Spring	A	INFO	no	buenos logs de negocio (uuid en msg), ops FTP/archivo
iuc-app-backend-v1	iuc	Spring	A	DEBUG	no	ruido /health DEBUG
adminntt-service-consumer-sms-v1	admint-ntt	Spring	A	WARN	no	HikariCP "failed to validate connection" spam
adminntt-service-consumer-email-v1	admint-ntt	Spring	A	WARN	no	HikariCP spam (scheduling-1)
adminntt-service-notification-consumer-v1	admint-ntt	Spring	A	DEBUG	no	ruido /health DEBUG
payments-processpaymentproducer-v1	integraciones	Spring	B	INFO	UTI en msg	Hibernate SQL echo ON (verboso), logs de publicación RMQ
payments-getcustomergeneralinfo-v1	integraciones	Spring WebFlux	B	DEBUG	reqId [...]	ruido /health DEBUG
billing-v1	digital	Spring	A	INFO/WARN	no	buenos logs de negocio (marcadores emoji)
emailqueue-consumer-v1	notificationmanager	Spring	B	INFO	no	arranque/init Hibernate
smsqueue-consumer-v1	notificationmanager	Spring + LogEntry	A	INFO/ERROR	UTI (NO_CORR_ID)	SMPP, stack traces

App	Imagen / base	Stack	Formato	Niveles	Corr ID	Contenido / ruido dominante
mainconsumer-v1	notificationmanager	Spring + LogEntry	B	INFO/WARN/ERROR	UTI	flujo de negocio, stack traces
tytan-tecrep-consumer-v1	tecrep	Spring	B	INFO/ERROR	no	stack traces (requiere unión multilínea)
tigo-spring-base-v1	base sit-1.1.0	Spring	A + Filter	DEBUG	SÍ (generado)	base de referencia; corr_id + version=1.0.0
tigo-python-base-v1	base sit-2.0.0	Python/uvicorn	C	INFO	no	acceso uvicorn /health
tigo-php-base-v1	base	PHP	D	ninguno	no	solo log de ping /health
cr-app-frontend-v1	cash-register	nginx	E	n/a	no	acceso /health kube-probe
adminntt-app-frontend-v1	admint-ntt	nginx	E	n/a	no	acceso /health kube-probe
ivrprocessapi-v1	notificationmanager	Python/FastAPI	C	INFO	no	arranque uvicorn + aviso pydantic multilínea

Observaciones transversales

- **DEBUG es el nivel efectivo en varias apps Spring** pese a `LOG_LEVEL=INFO` — la variable no está conectada a logback (el root logger queda en DEBUG → spam de DispatcherServlet/Mapping de Spring).
- **El ruido de sondas /health es universal** (en todos los stacks), a menudo ~100% del volumen en servicios de consulta.
- **Sin JSON en ningún lado**; los timestamps difieren (Formato A con espacio vs Formato B ISO8601+offset); niveles/campos están en posiciones distintas por formato.
- **La correlación ya existe en dos lugares**: el `CorrelationIdFilter` de la base Spring nueva, y el `UTI` de los consumidores. Simplemente **no se adopta de forma consistente** ni **se extrae a un campo**.

6. Estrategia de normalización (sin cambios de código de aplicación)

Dos vías complementarias. L1 es inmediata y libre de reconstrucción; L2 es la corrección duradera y aun así **no toca código fuente** (solo las imágenes base compartidas + una reconstrucción de CI).

Vía L1 — Normalización en el pipeline (fluent-bit, sin reconstrucción)

Se ejecuta enteramente en el colector; las apps no se tocan. **Es la parte que GLX no tiene** — SAT parte del esqueleto de GLX (input CRI + output ES + filtro k8s + allow-list + índice por namespace + la imagen 3.1.4 espejada) y agrega encima:

- **Parsers (uno por formato A-E)** para extraer `@timestamp`, `level`, `logger`, `thread`, `corr_id` / `UTI` y `message` a campos reales.
- **Multilínea** para unir los stack traces de Java (líneas de continuación: espacios iniciales, `at`, `Caused by`, `...`, `~` `[]`) en un solo evento — aplica a A y B.
- **Descarte de ruido de sondas** (líneas `/health`) en todos los formatos → índices mucho más pequeños y legibles.
- **Enriquecimiento** con metadata de k8s (namespace, pod, contenedor, label de app); **ruteo** a ES; **índice por namespace** `fluent-bit-sat-dev-<ns>` (patrón GLX).
- **Decisión necesaria** sobre el spam WARN de **HikariCP**: es una *señal real de fiabilidad* (la BD/firewall cierra conexiones inactivas antes de `maxLifetime`), por lo que la recomendación es **conservarlo pero corregir la causa raíz**, no descartarlo en silencio.

Límites de L1: puede parsear y descartar, pero no puede reducir lo que la app *emite* (las apps en DEBUG siguen inundando el nodo) ni inventar campos de negocio. Hace que los logs sean buscables y limpios en Kibana, pero la fuente sigue ruidosa.

Vía L2 — Estandarización de imagen base (reconstrucción, sin cambiar código fuente)

Como el logging está centralizado en `tigo-spring-base` / `tigo-python-base` / `tigo-php-base`, un solo cambio por imagen base se propaga a todas las apps derivadas en su próximo build de pipeline:

- **Emitir JSON** (p. ej. `JsonEncoder` de logback / formateador estructurado) → logging estructurado nativo de punta a punta, sin regex de pipeline.
- **Nivel INFO por defecto** y conectar realmente `LOG_LEVEL` → elimina la inundación DEBUG en la fuente.
- **Excluir** `/health` del logging de acceso/dispatcher → quita el ruido de sondas en la fuente.
- **Asegurar** `CorrelationIdFilter` en todas las bases (Spring ya lo tiene; agregar equivalentes a Python/PHP) y **loguear** `corr_id` / `UTI` como campo.

Límites de L2: requiere reconstruir/redesplegar imágenes (CI), y la adopción depende de que las apps actualicen su tag de base (varias aún corren versiones de base antiguas).

Secuenciación recomendada

1. **L1 ahora** (libre de reconstrucción) → los logs llegan suficientemente estructurados y limpios a Kibana de inmediato.
2. **L2 en paralelo** sobre las imágenes base → a medida que las apps reconstruyen, pasan a JSON nativo y el regex L1 de ese formato se vuelve redundante (se conserva L1 como respaldo para las apps aún no reconstruidas).

7. Objetivo propuesto para SAT dev (sin aplicar aún)

Clúster airgapped (confirmado): SAT **no tiene salida a Internet** (`download.elastic.co` / `docker.elastic.co` / `cr.fluentbit.io` inalcanzables). Todas las **imágenes y manifiestos de instalación** de ECK / Elasticsearch / Kibana / fluent-bit **deben espejarse al registro interno** (`registry-local.tigo.com.pa` / `git.tigopa.local:5050`) y aplicarse **offline** — sin `kubectl apply -f https://...`. El despliegue es GitOps vía `tigo-devops-panama` (ArgoCD), el repo único que ya usan todas las apps.

Hechos de entorno confirmados (SAT dev): StorageClass `nfs-storage` (default, `nfs-provisioner`, expansión habilitada) · IngressClass `nginx` (rke2-ingress-nginx) · **sin Prometheus** · repo GitOps `git.tigopa.local/kubernetes-panama/tigo-devops-panama.git`.

```
SAT dev (airgapped)
  registro interno → ECK (Elastic + Kibana, ns logging, PVC nfs-storage)
                    ▲
  fluent-bit DaemonSet —[L1: parsers + multilínea + drop /health]—┐
```

```
| índice por namespace fluent-bit-sat-dev-<ns>
└─ (Promtail→Loki en paralelo durante la transición)
```

```
tigo-*--base --[L2: JSON · INFO · sin /health · correlación]→ apps al reconstruir
```

Fase 1 — Captura + normalización

1. **Espejar imágenes** (operador ECK, Elasticsearch, Kibana, fluent-bit) al registro interno; vendorizar el manifiesto de instalación de ECK offline.
2. Desplegar **Elastic + Kibana propios** en el clúster vía **ECK** (ns `logging`; PVC en `nfs-storage`; `node.store.allow_mmap=false`), a través de ArgoCD/ `tigo-devops-panama`.
3. Desplegar **fluent-bit** (patrón GLX) → ES en clúster, con **L1** (parsers/multilínea/drop) e índice por namespace `fluent-bit-sat-dev-<ns>`.
4. Exponer Kibana vía Ingress (clase `nginx`).
5. Mantener **Promtail→Loki en paralelo** durante la transición (sin regresión).
6. Iniciar el trabajo **L2** de imagen base en paralelo (JSON, INFO, drop `/health`, correlación en todas) — punto de control único: las imágenes `tigo-*--base` + `tigo-devops-panama`.

Fase 2 — Habilitación de Kibana

- Data views (`fluent-bit-sat-dev-*`), dashboards por dominio (cr-*, iuc-*, adminntt-*, payments-*), ciclo de vida/retención de índices, RBAC.

Fase 3 — Istio + Kiali (greenfield, por etapas)

- Instalar istiod + Kiali **sin** inyectar ningún sidecar (cero impacto en apps).
- Requiere **Prometheus en SAT** para los grafos de tráfico de Kiali (hoy SAT solo tiene Loki/Grafana — por aprovisionar).
- Habilitar inyección **gradualmente**, primero apps de bajo riesgo; **excluir** ArgoCD/RabbitMQ; considerar mover las apps de negocio a un namespace dedicado primero.
- mTLS `PERMISSIVE` → `STRICT`. Bonus: el proxy puede estampar IDs de request/trace sin código de app, complementando §6.

8. Pendientes / datos por confirmar

Estado	Ítem
HECHO	Config fluent-bit de GLX: imagen <code>fluent-bit:3.1.4</code> (registro interno); input CRI; output ES a <code>elk-mnt.tigo.com.pa:443</code> (TLS, verify off, creds por env, <code>Logstash_Prefix</code> por namespace, <code>%Y.%m.%d</code>); parsers <code>cri / json / docker</code> solamente. Sin parser por app, sin multilínea → confirma que SAT debe agregar L1 encima.
HECHO	Pre-flight de SAT: StorageClass <code>nfs-storage</code> (default) · IngressClass <code>nginx</code> (rke2-ingress-nginx) · SIN salida a Internet → airgapped (imágenes deben espejarse al registro interno; instalación offline). Repo GitOps de todas las apps = <code>tigo-devops-panama</code> .
HECHO	Prometheus en SAT: ninguno presente (solo Loki/Grafana). Debe aprovisionarse para Kiali (Fase 3).
PENDIENTE	Confirmar si GLX prod corre Istio/Kiali. <i>Pendiente — requiere acceso a GLX prod (no disponible desde GLX UAT); consultar al equipo de plataforma.</i>
PARCIAL	Propiedad de imagen base / CI (Vía L2): todos los workloads — incluidos <code>tigo-spring-base-v1</code> , <code>tigo-python-base-v1</code> , <code>tigo-php-base-v1</code> — se despliegan desde el repo GitOps único <code>git.tigopa.local/kubernetes-panama/tigo-devops-panama.git</code> . El dueño del pipeline de build de imagen base / la cadencia de release sigue siendo una confirmación humana con el equipo de plataforma/CI.

